

DESARROLLO ASISTIDO POR INTELIGENCIA ARTIFICIAL DE UN RESOLVEDOR DE SISTEMAS DE ECUACIONES COMPLEJAS PARA ANÁLISIS DE CIRCUITOS

AI-ASSISTED DEVELOPMENT OF A COMPLEX SYSTEM SOLVER FOR CIRCUIT ANALYSIS

Orlando Contreras Reyes

Universidad Autónoma de Aguascalientes

al348390@edu.uaa.mx

Iker Emiliano García Barrera

Universidad Autónoma de Aguascalientes

al307630@edu.uaa.mx

Dr. Luís Alejandro Flores Oropeza

Universidad Autónoma de Aguascalientes

alejandro.floreso@edu.uaa.mx

Dr. Alejandro Román Loera

Universidad Autónoma de Aguascalientes

alejandro.roman@edu.uaa.mx

Resumen

Como proyecto integrador, se desarrolló una calculadora capaz de resolver sistemas de ecuaciones con números complejos, con el objetivo de facilitar los cálculos en métodos de análisis de circuitos como mallas o nodos. El programa se implementó en Python utilizando diversas librerías especializadas, con apoyo de herramientas de inteligencia artificial para acelerar el desarrollo. Además, se compiló como un ejecutable con interfaz gráfica compatible con los sistemas operativos Windows y GNU/Linux. La calculadora permite visualizar los resultados de forma clara e intuitiva para el usuario.

Palabras clave: Calculadora, Manual, Números complejos, Python, Sistema de ecuaciones.

Abstract

As an integrative project, a calculator was developed to solve systems of equations with complex numbers, with the goal of facilitating calculations in circuit anal-

ysis methods such as mesh and nodal analysis. The program was developed in Python using various specialized libraries and with the support of AI tools to accelerate development. It was implemented as an executable with a graphical user interface compatible with Windows and GNU/Linux. The calculator allows users to visualize results in a clear and intuitive manner.

Keywords: Calculator, Complex numbers, Manual, Python, System of equations.

1 Introducción

El análisis de circuitos es una de las tareas más importantes para un ingeniero, implica aplicar conceptos matemáticos y físicos para obtener parámetros como el voltaje o la corriente en un circuito. El análisis de circuitos es una aplicación directa de la ingeniería y gracias a él está formada la tecnología actual. Sin el análisis de circuitos no sería posible construir los dispositivos y las comodidades que se presentan en nuestro día a día.

Se reitera el concepto de practicidad y herramientas, pues en la labor del ingeniero es fundamental resolver problemas de forma eficiente. Es aquí donde entra el concepto de calculadora, una herramienta que permite la resolución de problemas matemáticos de forma eficiente. El ingeniero debe ser capaz, entonces, de crear herramientas que faciliten estos procesos. El problema que suele ocurrir con esta filosofía es la nula aplicación en el aula. En este caso, el problema que se abordó fue la necesidad de una solución para resolver circuitos de CA de forma efectiva y portátil. Las soluciones que han surgido a lo largo de los años para este problema son las siguientes:

- Comprar una calculadora específica que permita resolver este tipo de sistemas de ecuaciones complejas.
- Utilizar simuladores de calculadoras en el celular.
- Utilizar métodos de resolución, como la regla de Cramer, y aplicarlos manualmente.

La calculadora que usualmente se simulaba era la HP 48G tal y como se ve en la imagen 1 y su versión en aplicación 2, como se puede ver es una calculadora bastante robusta y con múltiples funciones, además teniendo botones que siempre están ahí a pesar de no ser útiles para lo que se está haciendo.

Los problemas encontrados en estas soluciones fueron los siguientes:

- Regla de Cramer: método poco eficiente.



Fuente: Wikipedia/HP-48

Figura 1 Calculadora HP-48



Fuente: Google Play Store/Droid 48

Figura 2 Droid 48

- Simuladores de calculadoras y calculadoras físicas: método de ingreso de datos poco eficiente, y obtención de resultados solo en forma rectangular o solo en forma polar.
- Deshonestidad académica Uso de calculadoras o simuladores para resolver de forma deshonesto, apoyándose en IA o en compañeros de clase, sin comprender el proceso.

Como se puede ver en la figura ?? la interfaz es poco amigable para el usuario, no se consigue aprovechar el tamaño de la pantalla dejándolo en una cantidad de celdas fijas y estática.

Es por ello que un equipo de estudiantes se dedicó a atender estas problemáticas directamente mediante la creación de una aplicación de escritorio, empaquetada en un ejecutable (.exe) como primer prototipo, para después implementarla como aplicación de Android y una versión en hardware.

Cabe destacar que el primer prototipo (.exe) fue diseñado en conjunto con

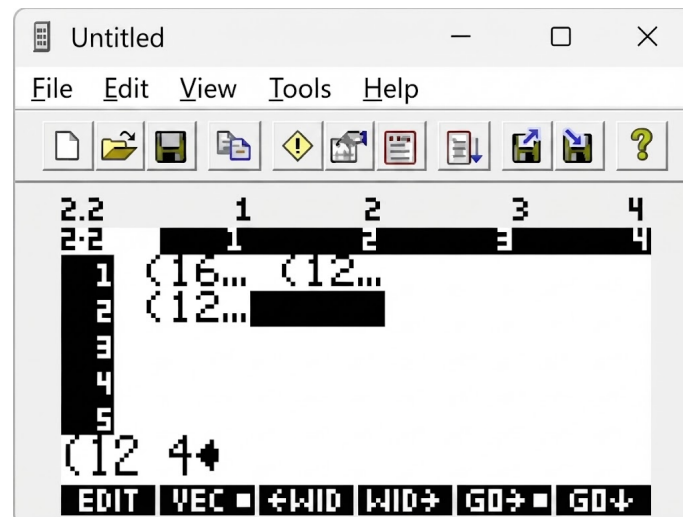


Figura 3 Solución de ecuaciones por medio de Droid48

Fuente: elaboración propia

la IA. Es importante destacar esto, pues demuestra el alcance de un ingeniero electrónico apoyado en inteligencia artificial:

2 Métodos

Para el desarrollo del proyecto se adoptó un enfoque metodológico dividido en tres fases principales: la selección de herramientas y arquitectura del software, la implementación de los algoritmos matemáticos en conjunto con el flujo de trabajo apoyado por Inteligencia Artificial, y la construcción de la interfaz gráfica junto con su posterior empaquetado ejecutable.

2.1 Materiales y Herramientas

- **Entorno de desarrollo:** Visual Studio Code v1.85+.
- **Lenguaje de programación:** Python 3.10+.
- **Librerías principales:**
 - NumPy: Manipulación de matrices y resolución de sistemas de ecuaciones lineales complejas mediante álgebra lineal numérica.
 - CustomTkinter y Tkinter: Creación y personalización de la interfaz gráfica de usuario (GUI) moderna.
 - Pillow (PIL): Procesamiento y renderizado de recursos gráficos dentro de la interfaz.
 - OS y JSON: Gestión de archivos del sistema, persistencia de datos e importación/exportación de historiales de matrices.

- **Herramientas de IA:** Asistente de código basado en el modelo ChatGPT (OpenAI), empleado para optimización de fragmentos de código, refactorización y aceleración en el diseño de componentes GUI.
- **Herramienta de compilación:** PyInstaller, para empaquetar el script de Python en un ejecutable binario autónomo (.exe).

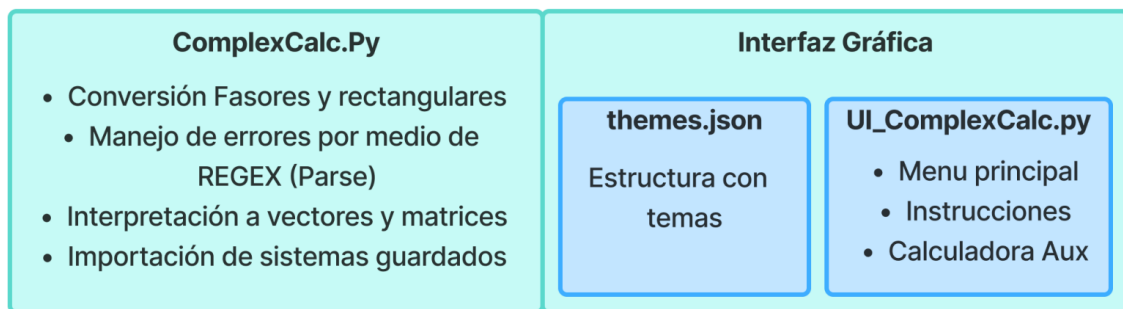


Figura 4 Arquitectura

Fuente: Elaboracion propia

2.2 Arquitectura del Software y Algoritmos Matemáticos

La herramienta está estructurada bajo una arquitectura modular dividida en lógica matemática, interfaz de usuario y gestión de datos:

1. **Procesamiento e ingreso de datos complejos:** Se diseñó un módulo capaz de recibir datos tanto en forma rectangular ($a + jb$) como polar ($r \angle \theta^\circ$).
2. **Resolución del sistema de ecuaciones:** Un sistema de mallas o nodos de $n \times n$ se representa matricialmente como:

$$[\mathbf{Z}][\mathbf{I}] = [\mathbf{V}] \implies [\mathbf{I}] = [\mathbf{Z}]^{-1}[\mathbf{V}] \quad (1)$$

donde $[\mathbf{Z}]$ es la matriz de impedancias complejas, $[\mathbf{I}]$ el vector de corrientes desconocidas y $[\mathbf{V}]$ el vector de fuentes. La resolución se realiza invocando el solver numérico de NumPy (`numpy.linalg.solve`), garantizando alta precisión decimal y eficiencia computacional frente al uso tradicional de la regla de Cramer.

3. **Conversión y renderizado de resultados:** Una vez calculado el vector solución, la herramienta convierte automáticamente cada componente a su forma polar ($r = \sqrt{a^2 + b^2}$, $\theta = \arctan(b/a)$) para presentar al usuario ambos formatos de manera simultánea e intuitiva.

2.3 Flujo de Trabajo Co-diseñado con Inteligencia Artificial

El desarrollo del software integró un ciclo iterativo de asistencia con IA (Chat-GPT) compuesto por cuatro etapas:

- **Generación de maquetas iniciales:** Solicitud de código base estructurado para ventanas, contenedores dinámicos y celdas matriciales editables.
- **Resolución de limitaciones de componentes:** Solución de errores al manipular arreglos multidimensionales complejos en tiempo real.
- **Refactorización y optimización:** Limpieza del código para separar la lógica de cálculo del renderizado gráfico, mejorando la mantenibilidad.
- **Supervisión y validación humana:** Cada segmento de código generado por la IA fue revisado, adaptado, probado técnicamente y validado contra ejercicios teóricos resueltos manualmente.

2.4 Diseño de la Interfaz Gráfica y Empaquetado

Para superar el aspecto nativo y limitado de Tkinter, se adoptó CustomTkinter complementado con un archivo de configuración `.json` para la paleta de colores y estilos visuales. Se incorporó una barra de herramientas con funciones para guardar matrices en disco, limpiar celdas y alternar la visualización del formato de salida.

Finalmente, el programa principal se compiló mediante PyInstaller utilizando los parámetros `--onefile` y `--noconsole`, empaquetando el intérprete de Python, las dependencias y los recursos estáticos en un solo ejecutable listo para su distribución.

2.5 Filosofía

Es importante destacar la filosofía con la que el proyecto se creó siendo la filosofía *Open Source* y *Vibe Coding* dos filosofías aparentemente opuestas pero muy fundamentales en la creación del proyecto.

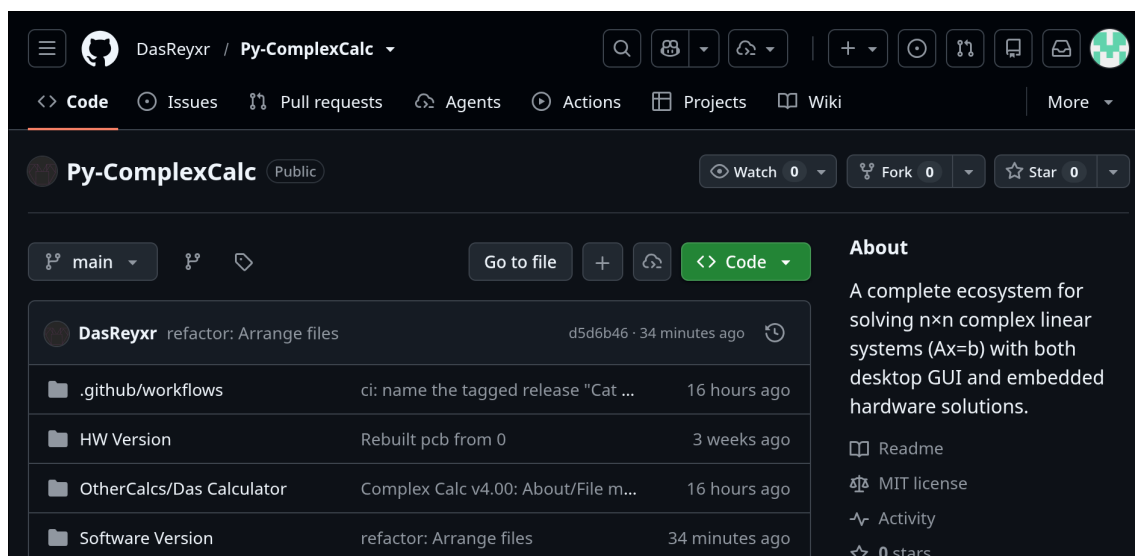
Este mismo busca la mejora a través de la comunidad y la accesibilidad al contribuir, es importante destacar que la creación del proyecto fue realizada por alumnos que sólo llevaron las bases de programación en C dedicado a microcontroladores, se destaca debido a el énfasis de la filosofía Vibe Coding, logrando que, con buenas bases en programación el ingeniero es capaz de poder construir sus herramientas para resolver necesidades de forma práctica; esta sinergia

permite desarrollar prototipos de hardware y software de forma óptima y práctica. Todo esto siguiendo una filosofía open source y open hardware, para facilitar la personalización y una reproducción accesible hacia el rubro que se está atendiendo: los ingenieros del mañana.

Parte de la filosofía del proyecto fue desarrollar una solución que pudiera estar disponible rápidamente, que fuera sencilla de utilizar y que, al mismo tiempo, tuviera una base tecnológica que permitiera demostrar su funcionamiento de manera práctica. Por esta razón, se optó por utilizar Python y apoyarse en herramientas de inteligencia artificial durante el proceso de desarrollo, buscando agilizar la implementación sin dejar de lado la funcionalidad del programa.

Asimismo, el proyecto se planteó como una forma de contribuir al salón y facilitar determinadas actividades académicas, particularmente durante los periodos de exámenes y en aquellas situaciones en las que una herramienta de este tipo pudiera resultar útil para los estudiantes.

De esta manera, el proyecto no se limita únicamente a resolver una necesidad inmediata, sino que busca servir como una base sobre la cual otros estudiantes puedan aprender, contribuir e incluso desarrollar sus propias soluciones. Se pretende que la sencillez del proyecto también pueda servir como inspiración para que más personas se animen a desarrollar herramientas similares y comprueben que la creación de software útil puede comenzar de una manera accesible y sencilla.



Fuente: Elaboración propia

Figura 5 Repositorio de Github de la calculadora

Aclaración de autoría y licenciamiento: Parte de las estructuras de código fuente e interfaz fueron desarrolladas con apoyo del modelo de inteligencia artificial ChatGPT. El proyecto se distribuye de manera pública bajo la licencia MIT.

3 Resultados

En la Figura 6 se muestra la solución obtenida por la calculadora para un sistema de ecuaciones complejas expresado en forma rectangular, donde se observan tanto los valores ingresados como el resultado calculado para cada incógnita.

The screenshot shows the 'Complex Calc 3.00' application window. The title bar indicates the theme is 'Pink'. The main interface includes a header with the logo and name 'Das Reyes • Iker Garcia'. Below this, there's a 'Solution history' section displaying the input equations: $x_1 = 1.387 \angle 3.18^\circ$ and $x_2 = 0.8771 \angle 37.87^\circ$, and the resulting solution: $x_1 = 2.557 \angle 6.911^\circ$ and $x_2 = 2.112 \angle 33.11^\circ$. The 'Saved systems' section is empty. On the left, there are input fields for complex numbers in rectangular form: $15j5$, -15 , 10 , -15 , $15j10$, and $-5j$. A 'Solve' button is present. At the bottom right, there is a calculator widget showing the result $6.667 + 5j$.

Fuente: elaboración propia

Figura 6 Solución de un sistema complejo de forma Rectangular.

En la Figura 7 se presenta una ecuación expresada en forma polar, lo que permite comprobar la conversión automática entre ambas representaciones que realiza la herramienta.

The screenshot shows the 'Complex Calc 3.00' application window with the same layout as Figure 6. The 'Solution history' section displays the input equations in polar form: $x_1 = 0.3008 \angle -0.9322^\circ$ and $x_2 = 0.3766 \angle -14.67^\circ$, and the resulting solution: $x_1 = 2.558 \angle 6.92^\circ$ and $x_2 = 2.113 \angle 33.12^\circ$. The 'Saved systems' section shows a saved system from 2026-08-20 19:19:08. The input fields on the left now contain polar values: $15.81 \angle 18.435^\circ$, $-15 \angle 0^\circ$, $10 \angle 0^\circ$, $-15 \angle 0^\circ$, $18.028 \angle -33.69^\circ$, and $-5 \angle 90^\circ$. The calculator widget at the bottom right shows the result $15 + 5j$.

Fuente: elaboración propia

Figura 7 Solución de un sistema complejo de forma Polar.

4 Discusión

4.1 Justificación y aportación de la herramienta

Esta calculadora fue desarrollada con el propósito de facilitar la resolución de sistemas de ecuaciones lineales, tanto con números reales como complejos, y de servir como apoyo en actividades académicas relacionadas con el análisis de circuitos eléctricos. La herramienta permite representar resultados en forma rectangular y polar, lo cual es especialmente útil en cursos como Análisis de Circuitos, Electrónica, Señales y Sistemas y Métodos Matemáticos para la Ingeniería, donde el manejo de cantidades fasoriales es fundamental.

Se incorporó un historial para conservar las operaciones realizadas y evitar la pérdida de información durante la resolución de ejercicios. Además, la interfaz fue diseñada para optimizar el tiempo de trabajo del usuario mediante una interacción clara y eficiente. Al ser una herramienta gratuita y ejecutarse en cualquier equipo con Windows / Linux, se convierte en una alternativa accesible dentro del ámbito académico, contribuyendo a la comprensión y visualización de magnitudes complejas mediante su conversión automática entre formas.

4.2 Proceso de diseño y desarrollo

Durante el desarrollo del proyecto se presentaron diversos retos. El primero consistió en el diseño de una interfaz lo suficientemente intuitiva para usuarios sin experiencia previa con herramientas de cálculo de números complejos. Las primeras versiones incluían instrucciones dentro de la ventana principal; sin embargo, esto afectaba la claridad visual, por lo que se realizó una reorganización estructural que permitió presentar la información de manera más ordenada y profesional.

Asimismo, se presentaron dificultades al implementar las funciones de almacenamiento y carga de matrices. Los errores iniciales durante las pruebas fueron corregidos mediante ajustes en la gestión de archivos y memoria, garantizando así un manejo estable de la información ingresada por el usuario.

Finalmente, se detectaron limitaciones en la personalización visual de Tkinter, dado que la aplicación de temas y combinaciones de colores no es directa. Para alcanzar la estética requerida fue necesario modificar de forma manual el archivo `.json` correspondiente, lo que permitió obtener una interfaz más coherente con el propósito académico del proyecto.

5 Conclusiones

El desarrollo de este proyecto permitió comprobar que las herramientas de inteligencia artificial pueden ser útiles durante la programación si se utilizan como apoyo y no como sustituto del conocimiento del programador. A pesar de contar únicamente con conocimientos básicos de C para microcontroladores, fue posible desarrollar una aplicación funcional para resolver sistemas de ecuaciones con números complejos, aplicados al análisis de circuitos. La revisión y las pruebas realizadas durante el desarrollo fueron necesarias para comprobar que el código generado funcionara correctamente y se ajustara a lo que requería el proyecto.

En cuanto al resultado técnico, se obtuvo una herramienta que permite trabajar con sistemas de ecuaciones de una forma más práctica que algunos de los métodos utilizados inicialmente. La aplicación permite introducir números complejos tanto en forma rectangular como polar y realizar la conversión entre ambas representaciones utilizando NumPy. También se consiguió generar un ejecutable que facilita su distribución y uso en diferentes equipos, evitando que sea necesario configurar todo el entorno de programación para utilizar la herramienta.

Durante el desarrollo también se reforzaron conocimientos relacionados con el álgebra lineal y su aplicación a circuitos eléctricos, principalmente en la representación de sistemas mediante matrices y en la utilización de matrices de impedancias. Una de las principales conclusiones del proyecto fue que utilizar IA para generar o modificar partes del código puede reducir considerablemente el tiempo de desarrollo, pero es necesario entender lo que hace el programa y comprobar cada parte antes de incorporarla. De esta forma, la IA funciona como una herramienta de apoyo, mientras que las decisiones y la validación del resultado siguen dependiendo del desarrollador.

Finalmente, el proyecto se publicó utilizando una licencia MIT, por lo que el código puede ser utilizado, estudiado y modificado por otros estudiantes. Aunque la aplicación surgió para resolver una necesidad específica dentro del curso, su estructura permite utilizarla como punto de partida para realizar mejoras o desarrollar herramientas similares. Esto también permite que el proyecto tenga una utilidad más allá de su entrega y que pueda seguir desarrollándose posteriormente.

Referencias

- [1] Anton, H., Rorres, C. *Álgebra lineal con aplicaciones*. Wiley, 2013.
- [2] Lay, D. C. *Álgebra lineal y sus aplicaciones*. 4.^a ed., Pearson, 2012.
- [3] NumPy Developers. *numpy.linalg.solve* — *NumPy v2.5 Manual*. NumPy Documentation, 2026. Disponible en: <https://numpy.org/doc/stable/reference/generated/numpy.linalg.solve.html>
- [4] Python Software Foundation. *Built-in Types* — *Numeric Types*. Python Documentation, 2026. Disponible en: <https://docs.python.org/3/library/stdtypes.html>
- [5] Python Software Foundation. *tkinter* — *Python interface to Tcl/Tk*. Python Documentation, 2026. Disponible en: <https://docs.python.org/3/library/tkinter.html>
- [6] Cortesi, D., Bajo, G., McMillan, W. *PyInstaller Manual*. PyInstaller Documentation, 2026. Disponible en: <https://pyinstaller.org/en/stable/>
- [7] OpenAI. *ChatGPT*. OpenAI, 2026. Disponible en: <https://chatgpt.com/>